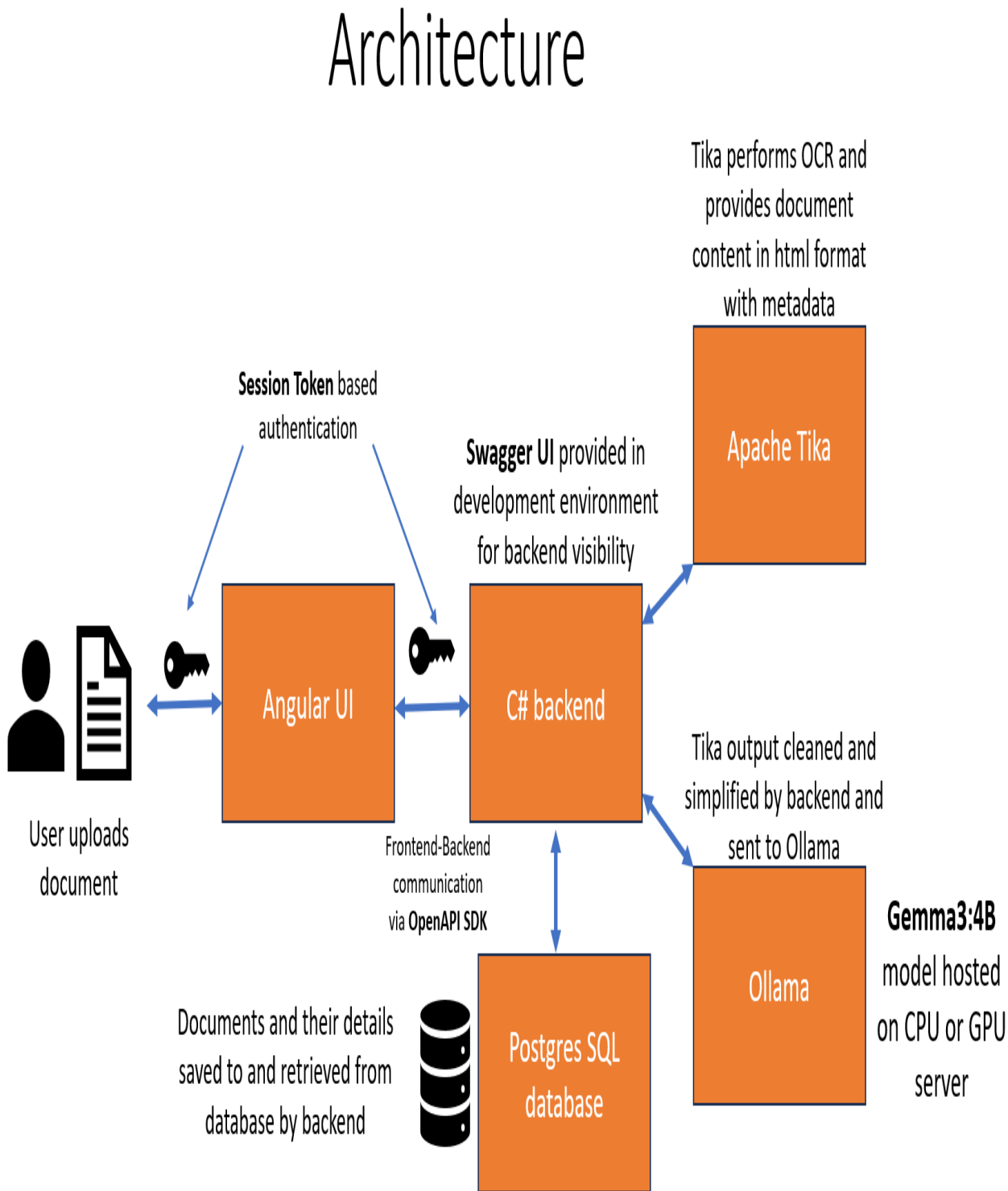


Technical Design Documentation

1. Project Overview

The Bit&Beam project is an intelligent document management system designed for building-related data. It aims to streamline document workflows using automated classification, metadata extraction, and intelligent search capabilities. The system supports multi-tenancy, ensuring data segregation and access control based on organizations.

2. System Architecture and CI/CD Overview



- **Session Management:**
Session tokens for both frontend and backend expire in 1 hour if not logged out.
- **Document Uploads:**
Uploaded documents can be PDFs or images.
- **Swagger UI:**
Available only in the development environment.
- **Service Deployment:**
Backend, frontend, Postgres, and Tika services are deployed using Docker Compose files in both development and production environments (separate compose files for each).
- **Production Access Control:**
Only the backend and frontend are accessible externally in production. Other services are accessible only internally by the backend via the Docker network.
- **Document Storage:**
Documents are stored in a dedicated Docker volume. Metadata (document/building details and links) is stored in a SQL database.
- **Ollama Deployment:**
The Ollama container (with LLM) is deployed separately to the production server using a different Docker Compose file. Separate Compose files are available for CPU-based and GPU-based servers. The Ollama service is externally accessible.
- **GitHub Workflows:**
 - Deploy containers to the production server on push to `main`
 - Deploy Ollama container on changes in Ollama configuration pushed to `main`
 - Run backend and frontend linting checks on push to `main` or PR to `main`
 - Run OpenAPI SDK generation on push to `main` or PR to `main`
 - **Note:** Linting and OpenAPI SDK generation workflows must pass for PRs to be merged into `main`, unless explicitly overridden

3. Key Components

- Frontend: Angular-based UI for user interaction.
 - Located in the [frontend/](#) directory.
 - Uses TypeScript and Angular CLI.
- Backend: C# ASP.NET Core Web API.
 - Located in the [backend/](#) directory.
 - Handles API requests, business logic, and data persistence.
- Database: PostgreSQL with pgai extension.
 - Configuration and schema in [database/](#) directory.
 - Search: Opensearch for indexing and full-text search.
- AI/Extraction:
 - Ollama for AI integration. Located in the [ollama/](#) directory.
 - Apache Tika for metadata extraction. See [TikaService.cs](#).

4. Data Model

The data model is defined using C# classes in the [backend/src/Models/](#) directory. Key entities include:

- User: System users information.
- Building: Stores building details.
- Document: Stores document metadata and file path.
- Organization: Represents a tenant.

5. Key Technologies

Programming Languages: TypeScript, C#, Python

Frameworks/Libraries: Angular, ASP.NET Core

Databases: PostgreSQL

AI systems: Ollama, Apache Tika

Containerization: Docker, Docker Compose

6. Development Guidelines

See [Coding_Guidelines.md](#)

- Protected main and dev branches.
- Feature branches.
- Pull requests require review from at least two other developers.

7. Build and Deployment

Docker Compose is used for orchestrating services. See [docker-compose.yml](#) and [docker-compose-prod.yml](#)

GitHub Actions workflows for linting and CI/CD. See [.github/workflows/](#)

8. Points to Note

- **Tika Output Preprocessing:**
Apache Tika's raw output is too noisy for LLMs in Ollama and significantly increases token count. Therefore, Tika output must be converted into clean, flat text before being passed to the LLM.
- **Model Selection:**
The **Gemma 3:4B** model provided the best trade-off between runtime and accuracy for both CPU-based and GPU-based hosting servers, among all major multilingual models available in the Ollama repository to date.
- **Ollama Configuration:**
Ollama LLM model configurations can be set through environment variables in the Docker Compose file for Ollama.